**BÁO CÁO TIẾN ĐỘ ĐỒ ÁN 2**

**Định tuyến mô hình ngôn ngữ theo từng bước cho tác tử Hermes Agent**

*Ứng dụng trong trợ lý phân tích dữ liệu qua Zalo/Telegram (tên làm việc: SCER-AH)*

| **Sinh viên thực hiện: Nguyễn Thành Duy** | **Giảng viên hướng dẫn: Nguyễn Đức Anh** |
|---|---|
| Ngày báo cáo: 04/11/2026 | Giai đoạn: xây dựng nền tảng, chuẩn bị thực nghiệm |

Kính gửi Thầy,

Em xin báo cáo tiến độ đồ án. Báo cáo tập trung trình bày **ý tưởng** của đề tài và những gì em đã xây dựng, để Thầy nắm được hướng đi và góp ý trước khi em hoàn thiện phần thuật toán. Thuật toán định tuyến hiện **đang trong thời gian phát triển** nên em chưa trình bày phương pháp cụ thể; các câu hỏi em đang cần làm rõ được nêu ở mục 4 và mục 7.


**Tóm tắt**

**Ý tưởng:** một tác tử AI giải một yêu cầu qua nhiều bước, nhưng hiện mọi bước đều dùng cùng một mô hình. Đề tài xây dựng một bộ định tuyến đặt bên trong vòng lặp của tác tử, chọn mô hình rẻ cho bước dễ và mô hình mạnh cho bước khó, dựa trên tình trạng thực thi thực tế (bước trước có lỗi không, đã có kỹ năng phù hợp chưa…).

**Mục tiêu:** giảm chi phí API và thời gian chờ, trong khi tỷ lệ hoàn thành tác vụ vẫn gần với cấu hình luôn dùng mô hình mạnh.

**Đã làm được:** trợ lý chạy thật trên Zalo/Telegram bằng Hermes Agent; ghi nhận chi tiết từng bước (mô hình, token, thời gian, lỗi, chi phí); trang quản trị Agent Console để quan sát và chạy benchmark; đã xác định cách gắn bộ định tuyến vào Hermes mà không phải sửa mã lõi.

**Đang phát triển:** thuật toán ra quyết định của bộ định tuyến; bộ đánh giá dựa trên các benchmark công khai đã được công bố trong các bài báo (InfiAgent-DABench, DA-Code, DSBench, MatPlotBench), bổ sung một bộ nhỏ tác vụ tiếng Việt.

**Xin ý kiến Thầy:** trọng tâm đóng góp, phạm vi thực nghiệm và tiêu chí đánh giá (mục 7).


## 1. Bối cảnh và bài toán

**Tác tử AI (AI agent)** khác chatbot thông thường ở chỗ tự thực hiện nhiều bước để hoàn thành một yêu cầu: lập kế hoạch, gọi công cụ (đọc file, chạy code, tìm kiếm), xem kết quả rồi quyết định bước tiếp theo. Mỗi bước là một lần gọi mô hình ngôn ngữ lớn (LLM). Với yêu cầu phân tích dữ liệu, một lần xử lý thường đi qua khoảng 4–10 bước như vậy.

Em sử dụng **Hermes Agent**, một framework tác tử mã nguồn mở của Nous Research, có sẵn môi trường chạy code (sandbox), cơ chế lưu kỹ năng (SKILL.md) và hệ thống plugin. Hermes cho phép gán mô hình riêng cho các việc phụ như nén ngữ cảnh hay đọc ảnh, nhưng **vòng lặp chính dùng một mô hình cố định cho mọi bước** của phiên; muốn đổi thì người dùng phải đổi bằng tay.

Điều này dẫn đến một đánh đổi khó chịu:

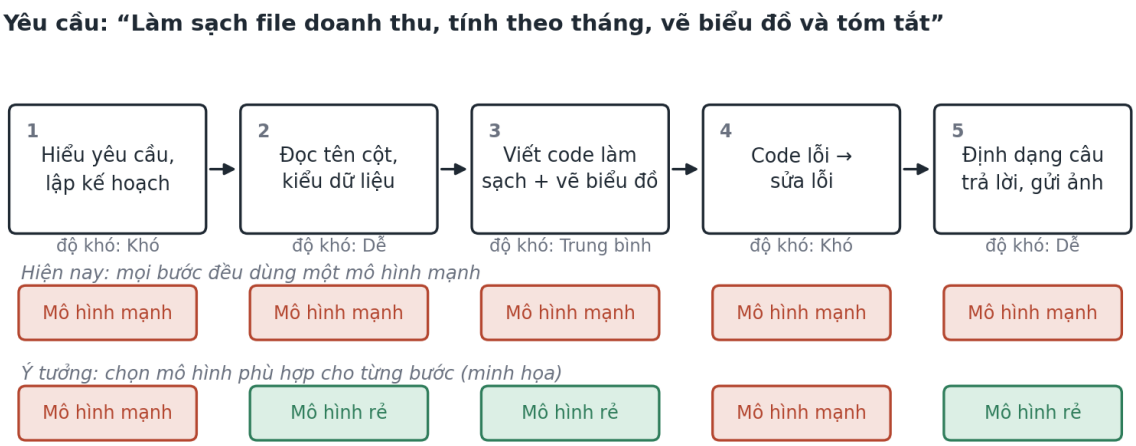
- Chọn **mô hình mạnh** cho cả phiên: bước đơn giản (đọc tên cột, định dạng câu trả lời) cũng chịu giá cao và chậm.
- Chọn **mô hình rẻ** cho cả phiên: bước khó (lập kế hoạch, sửa lỗi logic) dễ thất bại, phải thử lại nhiều lần hoặc bỏ dở.
- Trên nền tảng nhắn tin, người dùng chờ phản hồi theo thời gian thực, nên cả **chi phí** lẫn **độ trễ** đều quan trọng.

> **Câu hỏi nghiên cứu:** Có thể tự động chọn mô hình cho từng bước trong vòng lặp của tác tử để giảm chi phí và độ trễ, trong khi tỷ lệ hoàn thành tác vụ vẫn gần với khi luôn dùng mô hình mạnh hay không? Và những thông tin nào bên trong tác tử giúp ra quyết định đó tốt hơn?

## 2. Ý tưởng chính của đề tài

### 2.1. Quan sát: các bước trong cùng một yêu cầu khó dễ rất khác nhau

Hình 1 lấy ví dụ một yêu cầu phân tích file doanh thu. Bước lập kế hoạch và bước sửa lỗi cần khả năng suy luận tốt, trong khi đọc cấu trúc file hay định dạng câu trả lời là việc mô hình nhỏ cũng làm được. Hiện nay cả năm bước đều trả giá của mô hình mạnh.

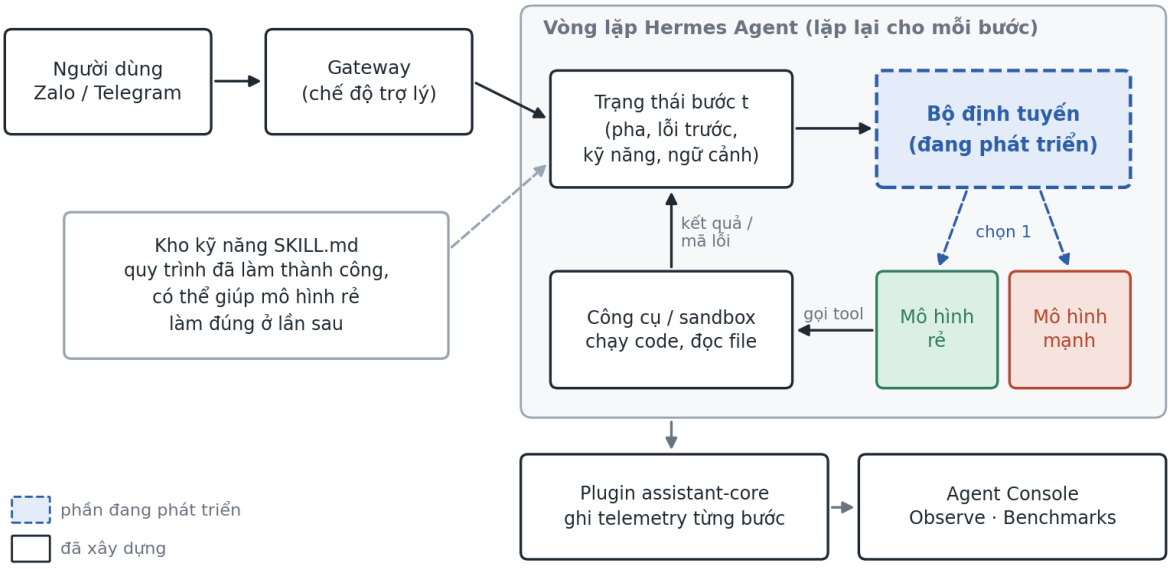



*Hình 1. Một yêu cầu đi qua nhiều bước có độ khó khác nhau. Mức độ khó và lựa chọn mô hình ở hàng dưới chỉ mang tính minh họa.*

Nếu một phần đáng kể số bước có thể giao cho mô hình rẻ mà kết quả cuối không đổi, chi phí và thời gian chờ sẽ giảm. Tỷ lệ đó thực tế là bao nhiêu là con số em sẽ **đo trong giai đoạn chạy thử**; hiện em chưa có số liệu để khẳng định.

### 2.2. Bộ định tuyến đặt bên trong vòng lặp của tác tử

Trước mỗi lần Hermes gọi mô hình, bộ định tuyến đọc **trạng thái hiện tại của tác tử** rồi chọn mô hình cho bước đó (Hình 2). Điểm khác với các cách định tuyến thông thường là bộ định tuyến không chỉ đọc câu hỏi của người dùng, mà nhìn vào những gì đang diễn ra bên trong tác tử: bước trước chạy code có lỗi không, đã có kỹ năng cho dạng việc này chưa, hội thoại đã dài đến đâu.




*Hình 2. Vị trí của bộ định tuyến trong hệ thống. Khối nét đứt là phần đang phát triển; các khối còn lại đã được xây dựng.*

Các nhóm thông tin em dự kiến đưa vào xem xét:

| Nhóm thông tin | Ví dụ | Vì sao có thể hữu ích |
|---|---|---|
| **Loại bước đang làm** | **Lập kế hoạch, viết code, sửa lỗi, trả lời** | **Mỗi loại việc đòi hỏi năng lực khác nhau** |
| Kết quả thực thi bước trước | Code chạy thành công hay báo lỗi; gọi công cụ sai tham số | Lỗi là tín hiệu rõ ràng rằng bước tiếp theo khó hơn bình thường |
| Kỹ năng đã có | Đã có quy trình SKILL.md cho dạng việc tương tự hay chưa | Có hướng dẫn sẵn thì mô hình rẻ dễ làm đúng hơn |
| Ngữ cảnh và mô hình hiện tại | Độ dài hội thoại, mô hình đang dùng ở bước trước | Việc đổi mô hình giữa chừng có chi phí riêng (mục 2.3) |

Cách kết hợp các nhóm thông tin này thành một quyết định cụ thể (thuật toán định tuyến) **đang trong thời gian phát triển**. Em sẽ trình bày phương pháp và lý do lựa chọn ở báo cáo sau, khi đã có dữ liệu chạy thử để so sánh và kiểm chứng.

### 2.3. Những yếu tố khiến bài toán không đơn giản

Khi tìm hiểu, em nhận thấy ba điểm khiến việc "cứ bước dễ thì dùng mô hình rẻ" không đủ, và đây cũng là phần em cho là có giá trị nghiên cứu:

- **Đổi mô hình có cái giá riêng.** Nhà cung cấp API lưu đệm phần đầu hội thoại (*prompt cache*) để các lượt sau đọc lại với giá rẻ hơn. Khi chuyển sang mô hình khác, bộ đệm này mất và toàn bộ ngữ cảnh bị tính giá đầy đủ. Vì vậy chuyển sang mô hình rẻ không phải lúc nào cũng rẻ hơn, nhất là khi hội thoại đã dài.
- **Sai một bước có thể kéo theo cả tác vụ.** Mô hình rẻ làm sai ở bước giữa có thể khiến các bước sau phải sửa lỗi, tốn hơn khoản tiết kiệm được. Do đó cần đánh giá theo kết quả cuối cùng của cả tác vụ, không chỉ từng bước riêng lẻ.
- **Không có sẵn dữ liệu "bước nào cần mô hình nào".** Để xây dựng và kiểm chứng bộ định tuyến, em phải tự tạo dữ liệu bằng cách chạy các mô hình trên cùng tác vụ và ghi lại kết quả từng bước. Đây là lý do phần ghi nhận từng bước được làm trước tiên.

### 2.4. Hướng mở rộng: kỹ năng tái sử dụng

Hermes có thể lưu một quy trình đã làm thành công thành file kỹ năng **SKILL.md**. Ý tưởng mở rộng là: khi mô hình mạnh giải xong một dạng việc mới, quy trình được đúc kết thành kỹ năng; lần sau gặp dạng tương tự, mô hình rẻ có thể làm theo kỹ năng đó. Theo thời gian, số lần cần gọi mô hình mạnh có thể giảm dần. Phần này phụ thuộc vào kết quả của bộ định tuyến chính và cần kiểm soát chất lượng, an toàn của kho kỹ năng, nên em đề xuất xem là hướng mở rộng và xin ý kiến Thầy (mục 7).

### 2.5. Liên hệ với các nghiên cứu liên quan

| Hướng tiếp cận | Cách làm | Khác biệt so với đề tài |
|---|---|---|
| **RouteLLM (Ong và cộng sự, ICLR 2025)** | **Học bộ phân loại chọn giữa mô hình mạnh và mô hình yếu cho mỗi câu hỏi** | **Quyết định một lần cho cả câu hỏi, không theo từng bước của tác tử** |
| FrugalGPT (Chen và cộng sự, 2023) | Gọi lần lượt từ mô hình rẻ đến đắt cho tới khi câu trả lời đủ tin cậy | Thiết kế cho truy vấn một lượt; với tác tử nhiều bước, gọi lại nhiều mô hình ở mỗi bước rất tốn |

| Hướng tiếp cận | Cách làm | Khác biệt so với đề tài |
|---|---|---|
| Jev Router (OpenRouter, 2026) | Dịch vụ thương mại chọn mô hình cho mỗi request, kể cả luồng agentic | Đứng ngoài tác tử như hộp đen, không thấy trạng thái nội bộ như kỹ năng hay kết quả sandbox |

Đóng góp em hướng tới là **đo được bằng thực nghiệm có đối chứng** lợi ích của việc dùng thông tin bên trong tác tử (kết quả thực thi, kỹ năng) và của việc tính đến chi phí chuyển mô hình khi định tuyến theo từng bước, thay vì tuyên bố đây là phương pháp đầu tiên.

## 3. Kết quả đã thực hiện

Em xây dựng theo thứ tự: trước hết là hệ thống chạy thật và khả năng quan sát từng bước, vì đây là điều kiện để thu dữ liệu và đánh giá bộ định tuyến sau này. Hệ thống phát triển tiếp từ bot Zalo của dự án em đã phát triển trước đó, chuyển sang vai trò trợ lý tổng quát.

| Hạng mục | Kết quả | Trạng thái |
|---|---|---|
| **Trợ lý trên Zalo/Telegram** | **Tin nhắn và ảnh của người dùng đi vào vòng lặp Hermes; trợ lý tìm thông tin, đọc ảnh, chạy code trong sandbox và gửi lại biểu đồ** | **Hoàn thành** |
| Triển khai cloud | Bot chạy trên Railway, trang quản trị trên Vercel; đã kiểm tra nhận/gửi tin với người dùng thật | Hoàn thành |
| Ghi nhận từng bước (telemetry) | Plugin assistant-core gắn vào các điểm móc của Hermes, ghi lại mỗi lần gọi mô hình và công cụ: tên mô hình, token vào/ra, token đọc từ cache, thời gian phản hồi, lỗi, chi phí ước tính. Không lưu nội dung tin nhắn | Hoàn thành |
| Lưu cấu hình thí nghiệm | Mỗi lần chạy được gắn nhãn và mã băm cấu hình (đã lọc khóa bí mật) để so sánh giữa các cấu hình | Hoàn thành |
| Agent Console | Trang quản trị web: xem từng lượt và từng bước của tác tử theo dòng thời gian; thống kê chi phí, độ trễ, tỷ lệ lỗi theo mô hình; so sánh các lần chạy | Hoàn thành |
| Bộ chạy benchmark | Chạy lặp lại một bộ tác vụ với các cấu hình khác nhau và tự chấm đạt/không đạt; hiện có 2 bộ mẫu, chấm dựa trên văn bản trả lời | Bản ban đầu |

| Hạng mục | Kết quả | Trạng thái |
|---|---|---|
| Điểm tích hợp bộ định tuyến | Đã khảo sát mã nguồn Hermes: có thể đổi mô hình cho từng lần gọi qua cơ chế middleware của plugin, không cần sửa mã lõi Hermes | Đã xác định |

**Ràng buộc kỹ thuật phát hiện khi khảo sát** (ảnh hưởng đến thiết kế sau này):

- Các mô hình được chọn nên dùng chung một nhà cung cấp/giao thức (ví dụ cùng qua OpenRouter), vì một số tham số gọi mô hình được Hermes chuẩn bị sẵn theo mô hình mặc định.
- Trong cùng một bước không thể gọi mô hình rẻ rồi gọi lại mô hình mạnh; nếu cần "nâng cấp" thì phải thực hiện ở bước kế tiếp.
- Khi bộ định tuyến gặp sự cố, Hermes giữ mô hình mặc định; em sẽ đặt mặc định là mô hình mạnh để lỗi của bộ định tuyến không làm giảm chất lượng trả lời.

Nhờ phần nền tảng này, mỗi lần chạy thử đều để lại dữ liệu chi tiết theo bước, là đầu vào để phân tích "bước nào cần mô hình mạnh" và để so sánh chi phí giữa các cấu hình. **Em chưa có kết quả định lượng nào về hiệu quả định tuyến.**

## 4. Nội dung đang phát triển

### 4.1. Thuật toán định tuyến

Đây là phần đang trong thời gian phát triển. Trước khi chốt phương pháp, em đang làm rõ các câu hỏi thiết kế sau:

- Những thông tin nào thực sự giúp dự đoán một bước có thể giao cho mô hình rẻ mà không ảnh hưởng kết quả cuối?
- Khi một bước gặp lỗi, nên phản ứng thế nào để sửa được lỗi mà không tốn quá nhiều lần thử?
- Khi nào việc chuyển mô hình là đáng, nếu tính cả chi phí mất bộ đệm ngữ cảnh?
- Làm thế nào để có đủ dữ liệu xây dựng và kiểm chứng bộ định tuyến trong giới hạn ngân sách API của sinh viên?

Các câu hỏi này cần dữ liệu từ giai đoạn chạy thử để trả lời. Em sẽ trình bày phương pháp cụ thể cùng lý do lựa chọn ở báo cáo tiếp theo.

### 4.2. Bộ tác vụ đánh giá và bộ chấm

Để kết quả đánh giá có độ tin cậy và so sánh được với các nghiên cứu khác, em chuyển từ hướng tự xây toàn bộ sang dùng các benchmark công khai đã được công bố tại hội

nghị uy tín làm phần đánh giá chính. Các bộ này có sẵn dữ liệu, đáp án kiểm chứng được và bộ chấm tự động do tác giả cung cấp, đồng thời đều có các tác vụ nhiều bước, phù hợp để đo hiệu quả định tuyến theo từng bước. Bộ tác vụ tự xây chỉ giữ ở quy mô nhỏ để kiểm tra trường hợp sử dụng thật qua Zalo:

| Nguồn | Nội dung | Cách chấm và vai trò |
|---|---|---|
| **InfiAgent-DABench (Hu và cộng sự, ICML 2024)** | **257 câu hỏi phân tích dữ liệu trên 52 file CSV, chia ba mức dễ/trung bình/khó: thống kê, tương quan, xử lý dữ liệu thiếu, phát hiện ngoại lai…** | **Đáp án dạng đóng theo định dạng quy định, chấm tự động bằng so khớp. Bộ đánh giá chính cho các tác vụ L1–L2** |
| DA-Code (Huang và cộng sự, EMNLP 2024) | 500 tác vụ khoa học dữ liệu dạng agent: làm sạch dữ liệu, EDA, học máy, trên dữ liệu thật và môi trường chạy code | Bộ chấm tự động của tác giả theo từng loại đầu ra (bảng, số, file). Đại diện cho tác vụ nhiều bước, nhiều lần gọi công cụ (L3) |
| DSBench (Jing và cộng sự, ICLR 2025) | 466 tác vụ phân tích dữ liệu (từ cuộc thi ModelOff) và 74 tác vụ mô hình hóa (từ Kaggle); dữ liệu Excel nhiều sheet, ngữ cảnh dài | So khớp đáp án và chỉ số của tác giả. Dùng mẫu con các bài khó để kiểm tra khi nào cần mô hình mạnh |
| MatPlotBench (Yang và cộng sự, ACL Findings 2024) | 100 yêu cầu vẽ biểu đồ có ảnh chuẩn đi kèm | Chấm tự động bằng mô hình thị giác so với ảnh chuẩn theo quy trình của bài báo. Đánh giá bước vẽ biểu đồ |
| Bộ tác vụ tiếng Việt tự xây (khoảng 30 tác vụ) | Yêu cầu mô phỏng tin nhắn Zalo: thống kê, làm sạch, vẽ biểu đồ, tóm tắt tiếng Việt | So khớp số liệu, kiểm tra file; tóm tắt chấm bằng LLM theo rubric. Kiểm tra tính khả dụng thực tế và dùng khi demo |

Do ngân sách API có hạn, em sẽ lấy mẫu con phân tầng theo mức độ khó của từng bộ (ví dụ khoảng 100 tác vụ mỗi bộ với DA-Code và DSBench), chia thành tập phát triển để điều chỉnh bộ định tuyến và tập kiểm tra giữ riêng chỉ dùng cho đánh giá cuối. Kết quả được báo cáo theo đúng chỉ số gốc của từng benchmark để đối chiếu được với số liệu đã công bố. Hiện bộ chạy benchmark trong Agent Console mới chấm theo văn bản trả lời; em đang viết bộ chuyển đổi để nạp các bộ trên và gọi lại bộ chấm gốc của tác giả, kèm kiểm tra số liệu (trong sai số cho phép) và file kết quả.

### 4.3. Mở rộng ghi nhận và hiển thị

Khi có bộ định tuyến, mỗi bước cần ghi thêm mô hình được chọn, thông tin trạng thái tại thời điểm chọn và kết quả của bước đó. Agent Console sẽ hiển thị quyết định định tuyến theo từng bước, giúp kiểm tra và giải thích hành vi của bộ định tuyến khi demo.

## 5. Kế hoạch đánh giá

Bộ định tuyến sẽ được so sánh với các cấu hình đối chứng trên cùng bộ tác vụ, cùng sandbox và cùng cặp mô hình:

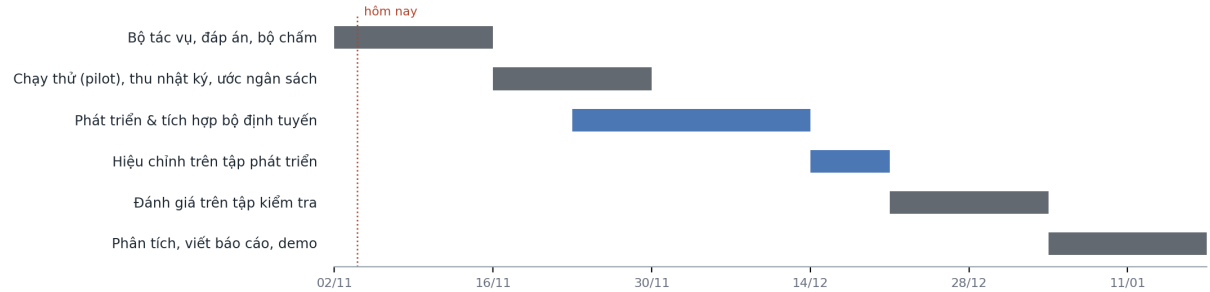
| Cấu hình | Mô tả | Vai trò |
|---|---|---|
| **Luôn dùng mô hình mạnh** | **Mọi bước dùng mô hình mạnh** | **Mốc chất lượng cao nhất** |
| Luôn dùng mô hình rẻ | Mọi bước dùng mô hình rẻ | Mốc chi phí thấp nhất |
| Luật đơn giản | Chọn theo loại bước, chuyển lên mô hình mạnh khi gặp lỗi | Kiểm tra bộ định tuyến có tốt hơn cách làm đơn giản không |
| Bộ định tuyến đề xuất | Phương pháp đang phát triển | Đối tượng nghiên cứu |
| Đối chứng bổ sung (nếu đủ ngân sách) | Jev Router; chọn ngẫu nhiên theo cùng tỷ lệ | So với dịch vụ thương mại; kiểm tra bộ định tuyến có thực sự học được gì |

Các tiêu chí dự kiến:

- **Tỷ lệ hoàn thành tác vụ: tính theo chỉ số gốc của từng benchmark (độ chính xác của InfiAgent-DABench, điểm của DA-Code, DSBench, MatPlotBench), báo cáo riêng từng bộ và theo mức độ khó.**
- **Chi phí trên mỗi tác vụ thành công:** tính cả chi phí của các lượt thất bại và thử lại, để tránh trường hợp cấu hình rẻ nhưng sai nhiều trông có vẻ tốt.
- **Thời gian hoàn thành:** trung bình và các trường hợp chậm nhất (p95).
- **Diễn biến xử lý:** số bước, số lần gặp lỗi, tỷ lệ bước giao cho mô hình rẻ, số lần chuyển mô hình.

Trước khi chạy trên toàn bộ tác vụ, em sẽ chạy thử trên một nhóm nhỏ để kiểm tra bộ chấm, ước tính chi phí và điều chỉnh quy mô. Các cấu hình chính sẽ được chạy lặp lại để báo cáo khoảng tin cậy, vì kết quả của LLM dao động giữa các lần chạy.

## 6. Kế hoạch thực hiện tiếp theo




*Hình 3. Kế hoạch dự kiến. Màu xanh là phần phát triển bộ định tuyến; thời gian sẽ điều chỉnh theo lịch của học phần và góp ý của Thầy.*

| Thời gian | Công việc | Kết quả cần đạt |
|---|---|---|
| **02/11 – 15/11/2026** | **Tích hợp InfiAgent-DABench, DA-Code, DSBench, MatPlotBench và bộ chấm gốc; xây bộ tác vụ tiếng Việt** | **Bộ đánh giá chạy được trên Agent Console; đã chia tập phát triển/kiểm tra** |
| 16/11 – 29/11/2026 | Chạy thử các cấu hình, thu nhật ký từng bước, ước tính ngân sách | Dữ liệu ban đầu; chốt quy mô thực nghiệm |
| 23/11 – 13/12/2026 | Hoàn thiện thuật toán, tích hợp bộ định tuyến vào Hermes | Phiên bản thử nghiệm, quan sát được từng bước |
| 14/12 – 20/12/2026 | Điều chỉnh trên tập phát triển | Cấu hình ổn định trước khi đánh giá |
| 21/12/2026 – 03/01/2027 | Đánh giá trên tập kiểm tra | Bảng số liệu chất lượng, chi phí, thời gian |
| 04/01 – 17/01/2027 | Phân tích, viết báo cáo, chuẩn bị demo | Báo cáo thực nghiệm và kịch bản demo |

**Việc cụ thể trong hai tuần tới:**

- Viết bộ chuyển đổi để nạp InfiAgent-DABench, DA-Code, DSBench, MatPlotBench vào bộ chạy benchmark và gọi bộ chấm gốc của tác giả.
- Lấy mẫu con phân tầng theo độ khó, chia tập phát triển/kiểm tra; viết khoảng 30 tác vụ tiếng Việt mô phỏng yêu cầu qua Zalo.
- Chọn cặp mô hình mạnh/rẻ cùng nhà cung cấp; cập nhật bảng giá có tính giá cache.
- Chạy thử hai cấu hình “luôn mạnh” và “luôn rẻ” trên 5–10 tác vụ để ước tính chi phí mỗi lượt.

### Rủi ro chính

| Rủi ro | Ảnh hưởng | Cách giảm |
|---|---|---|
| **Chi phí mất cache khi đổi mô hình lớn hơn khoản tiết kiệm** | **Định tuyến không giảm được chi phí thật** | **Đo riêng khoản này trong pilot; đưa vào tiêu chí quyết định** |
| Chi phí API vượt ngân sách | Không đủ số liệu | Ước tính từ pilot; cắt các đối chứng bổ sung trước |
| Giới hạn lượt dùng của gói ChatGPT | Chạy benchmark bị chậm hoặc gián đoạn | Chạy theo đợt, lưu kết quả từng tác vụ để chạy tiếp; dự phòng API key |
| Benchmark công khai có thể đã lọt vào dữ liệu huấn luyện của mô hình | Điểm tuyệt đối bị cao hơn thực tế | Tập trung so sánh tương đối giữa các cấu hình trên cùng bộ; đối chiếu thêm với bộ tiếng Việt tự xây |
| Kết quả LLM dao động giữa các lần chạy | Kết luận thiếu chắc chắn | Lặp các cấu hình chính, báo khoảng tin cậy |
| Hermes cập nhật làm thay đổi điểm tích hợp | Bộ định tuyến ngừng hoạt động | Cố định phiên bản Hermes khi thực nghiệm |

## 7. Nội dung em xin ý kiến Thầy

Em mong Thầy góp ý về các điểm sau để em định hướng phần thuật toán và thực nghiệm:

1. **Hướng ý tưởng:** định tuyến mô hình theo từng bước dựa trên trạng thái bên trong tác tử (kết quả thực thi, kỹ năng, chi phí chuyển mô hình). Thầy thấy hướng này có phù hợp và đủ tính mới để làm đóng góp chính của đồ án không?
2. **Kỹ năng tái sử dụng:** nên đưa vào phạm vi chính, hay để là hướng mở rộng sau khi bộ định tuyến chính có kết quả?
3. **Hình thức thuật toán:** khi hoàn thiện, Thầy khuyên em ưu tiên phương pháp đơn giản, dễ giải thích và kiểm chứng được với lượng dữ liệu nhỏ, hay có thể thử các hướng học máy phức tạp hơn?
4. **Đánh giá:** em dự kiến dùng InfiAgent-DABench, DA-Code, DSBench và MatPlotBench làm bộ đánh giá chính, kèm một bộ nhỏ tác vụ tiếng Việt. Thầy thấy lựa chọn này đã phù hợp chưa, có cần thay hoặc thêm benchmark nào khác không? Mức giảm tỷ lệ hoàn thành chấp nhận được khi đổi lấy chi phí thấp hơn là bao nhiêu (ví dụ không quá 5 điểm phần trăm)?

5. **Đối chứng thương mại:** có cần so sánh với dịch vụ định tuyến thương mại (Jev Router) không, hay so với các cấu hình tự xây là đủ?
6. **Nguồn lực và mốc:** phần lớn thực nghiệm sẽ chạy qua gói ChatGPT nên chi phí thực tế thấp, nhưng gói có giới hạn số lượt dùng theo thời gian, có thể làm chậm việc chạy lặp nhiều lần. Em xin Thầy góp ý mức ngân sách API bổ sung hợp lý (cho Claude hoặc khi hết hạn mức), và các mốc báo cáo của học phần em cần bám theo.

Sau khi nhận góp ý của Thầy, em sẽ điều chỉnh phạm vi, hoàn thiện thuật toán định tuyến và trình bày phương pháp chi tiết trong báo cáo tiến độ tiếp theo.

Em xin cảm ơn Thầy.

## Phụ lục: Giải thích thuật ngữ

| Thuật ngữ | Nghĩa trong báo cáo |
|---|---|
| **Tác tử (agent)** | **Chương trình dùng LLM tự thực hiện nhiều bước, gọi công cụ để hoàn thành một yêu cầu** |
| Bước | Một lần tác tử gọi mô hình trong vòng lặp; sau đó có thể gọi công cụ rồi sang bước tiếp |
| Định tuyến mô hình | Chọn mô hình nào sẽ xử lý một bước (hoặc một câu hỏi) |
| Sandbox | Môi trường riêng để chạy code do tác tử sinh ra và thu kết quả hoặc thông báo lỗi |
| SKILL.md | File mô tả quy trình làm một dạng việc, Hermes nạp vào ngữ cảnh khi cần |
| Token | Đơn vị tính độ dài văn bản; chi phí API tính theo số token vào/ra |
| Prompt cache | Bộ đệm phần đầu hội thoại ở phía nhà cung cấp, giúp các lượt sau đọc lại rẻ hơn; mất khi đổi mô hình |
| Telemetry | Bản ghi số liệu từng bước: mô hình, token, thời gian, lỗi, chi phí |
| Benchmark | Bộ tác vụ có đáp án dùng để chạy và chấm tự động các cấu hình |

## Tài liệu tham khảo


• Nous Research. *Hermes Agent* – mã nguồn và tài liệu “Configuring Models”.

- Ong, I. và cộng sự. *RouteLLM: Learning to Route LLMs with Preference Data.* ICLR 2025.
- Chen, L., Zaharia, M., Zou, J. *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance.* 2023.
- Hu, X. và cộng sự. InfiAgent-DABench: Evaluating Agents on Data Analysis Tasks. ICML 2024.
- Huang, Y. và cộng sự. DA-Code: Agent Data Science Code Generation Benchmark for Large Language Models. EMNLP 2024.
- Jing, L. và cộng sự. DSBench: How Far Are Data Science Agents from Becoming Data Science Experts? ICLR 2025.
- Yang, Z. và cộng sự. MatPlotAgent: Method and Evaluation for LLM-Based Agentic Scientific Data Visualization. Findings of ACL 2024.
- OpenRouter. *Jev Router* – tài liệu dịch vụ định tuyến mô hình.
- Đề xuất Đồ án 2 – SCER-AH cho Hermes Agent (31/10/2026).